چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱

چرخهٔ عمر Activity، مدیریت state و process death



اسماعیل مرادی

@esmaeeil-moradi700

در روز ۱ چه می‌خوانیم؟

هر جواب همان‌طور نوشته شده که یک مصاحبه‌گر سینیور دوست دارد بشنود:

- **جواب** — سازوکار ماجرا، دقیق و بی‌ابهام
- **تلهٔ مصاحبه** — جایی که بیشتر کاندیداها می‌لغزند
- **سؤال تکمیلی** — سؤالی که بلافاصله بعدش از شما می‌پرسند

مبنای ۲۰۲۶: Android 16 با targetSdk 36، Kotlin، Coroutines و Flow، Jetpack Compose، و نسخه‌های فعلی Lifecycle و Navigation در AndroidX. هر جا مصاحبه‌ها هنوز دربارهٔ Viewها و Fragmentها می‌پرسند، آن‌ها هم پوشش داده شده‌اند.

سطح	سؤال‌ها
جونیور	callbackها، configuration change، ذخیرهٔ state
مید	نشت حافظه، `onStop` در برابر `onDestroy`، Navigation و نتیجه‌ها، taskها
سینیور	کامپوننت‌های lifecycle-aware، زمان startup، بازگردانی state در چند صفحه
آرکیتکت	مانیتورینگ در کل اپ، streamهای real-time، بقای پروسه



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: stateها، نه callbackها

یک `Lifecycle` **۵ state** دارد. eventها آن را بین این stateها جابه‌جا می‌کنند:

```
INITIALIZED
    | ON_CREATE
    ▼
 CREATED ──ON_START──► STARTED ──ON_RESUME──► RESUMED
 CREATED ◄──ON_STOP─── STARTED ◄──ON_PAUSE─── RESUMED
    | ON_DESTROY
    ▼
DESTROYED
```

- `CREATED` — وجود دارد اما دیده نمی‌شود (بعد از `onCreate`، و دوباره بعد از `onStop`)
- `STARTED` — دیده می‌شود؛ شاید پشت یک Activity نیمه‌شفاف یا در حالت multi-window
- `RESUMED` — دیده می‌شود و تعامل‌پذیر است
- `DESTROYED` — حالت پایانی؛ دیگر هیچ eventی نمی‌آید

چرا مهم است

APIهای مدرن — `repeatOnLifecycle`، `collectAsStateWithLifecycle` و `LifecycleStartEffect` — بر اساس **state** تعریف شده‌اند. به «تا وقتی حداقل STARTED هستی کار کن» فکر کنید، نه «در `onResume` شروع کن و در `onPause` متوقف کن».



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

callbackها، configuration change، ذخیرهٔ state



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. lifecycle اکتیویتی را توضیح دهید

Callback	state بعد از آن	چه کاری اینجا انجام شود
`onCreate`	CREATED	`setContent` یا inflate، گرفتن ViewModel، خواندن `savedInstanceState`
`onStart`	STARTED	حالا دیده می‌شود. شروع کارهای وابسته به UI، مثل collect کردن flowها
`onResume`	RESUMED	تعامل‌پذیر. منابع انحصاری: دوربین، سنسورهای پرسرعت
`onPause`	STARTED	**سریع نگهش دارید.** کارهای وابسته به focus را متوقف کنید. در multi-window هنوز دیده می‌شود
`onStop`	CREATED	دیده نمی‌شود. توقف کارهای UI، آزاد کردن منابع سنگین، ذخیرهٔ پیش‌نویس‌ها
`onDestroy`	DESTROYED	پاک‌سازی نهایی. **اجرایش تضمینی نیست**

وقتی یک Activity متوقف‌شده دوباره به پیش‌زمینه برمی‌گردد، `onRestart` قبل از `onStart` اجرا می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. ترتیب مهم است: باز شدن B از A

```
A.onPause → B.onCreate → B.onStart → B.onResume → A.onStop
```

- B **قبل از** متوقف شدن A ساخته می‌شود، پس یک `A.onPause` کند، اولین فریم B را عقب می‌اندازد.
- اگر B نیمه‌شفاف باشد یا theme دیالوگ داشته باشد، A در حالت **STARTED** می‌ماند — `onStop` صدا زده نمی‌شود.
- برگشت از B: `B.onPause → A.onRestart → A.onStart → A.onResume → B.onStop → B.onDestroy`

تلهٔ مصاحبه

یک `AlertDialog` یا `DialogFragment` اکتیویتی را pause **نمی‌کند** — فقط یک window در همان Activity است. فقط یک *Activity* دیگر (یک Activity با theme دیالوگ، یا پنجرهٔ درخواست runtime permission) باعث `onPause` می‌شود.

تلهٔ مصاحبه

هیچ‌وقت منطقی را که حتماً باید ذخیره شود فقط در `onDestroy` نگذارید. وقتی سیستم پروسهٔ شما را kill می‌کند، `onDestroy` اصلاً صدا زده نمی‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. در configuration change چه اتفاقی می‌افتد؟

به‌طور پیش‌فرض Activity **از بین می‌رود و دوباره ساخته می‌شود** تا resourceها برای پیکربندی جدید دوباره load شوند:

```
onPause → onStop → onSaveInstanceState → onDestroy
            ── new instance ──
onCreate(bundle) → onStart → onRestoreInstanceState → onResume
```

- عامل‌ها: چرخش صفحه، تغییر اندازهٔ پنجره (multi-window، دستگاه‌های تاشو، desktop windowing)، زبان، تم تیره، اندازهٔ فونت، density و کیبورد.
- در `onStop` و `onDestroy`، مقدار `isChangingConfigurations` برابر `true` است.
- از API 28، `onSaveInstanceState` **بعد از** `onStop` اجرا می‌شود (قبل از API 28، پیش از `onStop`).
- `ViewModelStore` باقی می‌ماند، چون `ComponentActivity` آن را از طریق `onRetainNonConfigurationInstance` به نمونهٔ جدید می‌دهد — در حافظه، داخل همان پروسه.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چه چیزی در ۲۰۲۶ عوض شده

ANDROID 16 (API 36)

روی صفحه‌های بزرگ (کوچک‌ترین عرض ۶۰۰dp یا بیشتر)، در اپ‌هایی که API 36 را target می‌کنند محدودیت‌های `screenOrientation`، `resizeableActivity` و نسبت تصویر **نادیده گرفته می‌شوند** (به‌جز بازی‌ها). «قفل کردن روی حالت عمودی» دیگر راهی برای فرار از recreate شدن نیست — مدیریت درست state الزامی است.

تلهٔ مصاحبه

`android:configChanges` فقط برای تغییرهایی که فهرست شده‌اند جلوی recreate شدن را می‌گیرد. برای process death **هیچ کاری** نمی‌کند، و خودتان باید resourceها را از `Configuration` جدید دوباره بخوانید. در اپ‌هایی که تماماً Compose هستند می‌تواند یک انتخاب آگاهانه باشد؛ در اپ‌های View-based معمولاً بوی بد کد است.

سؤال تکمیلی

«چرا ViewModel از چرخش صفحه عبور می‌کند ولی از process death نه؟» — چون در حافظه نگه داشته می‌شود و به نمونهٔ جدید تحویل داده می‌شود. process death حافظه را پاک می‌کند؛ فقط `Bundle` مربوط به saved state، که سیستم نگهش می‌دارد، برمی‌گردد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. state را چطور حفظ می‌کنید؟

اول نوع state را مشخص کنید — ابزار خودش معلوم می‌شود:

نوع state	مثال	کجا نگه داشته می‌شود
عنصر UI	اسکرول، متن ورودی، کارت بازشده	`rememberSaveable` یا saved state ویو
صفحه	لیست loadشده، حالت loading یا error	`ViewModel` که `StateFlow` منتشر می‌کند
کلیدهای بازسازی	id آیتم، query، تب انتخاب‌شده	`SavedStateHandle`
دادهٔ اپ	کاربر، سبد خرید، پیش‌نویس‌ها	Room یا DataStore، به‌علاوهٔ شبکه

چه چیزی باقی می‌ماند؟	چرخش صفحه	process death	بستن task توسط کاربر
فیلد ViewModel	بله	خیر	خیر
saved state: `Bundle`، `SavedStateHandle`، `rememberSaveable`	بله	بله	خیر
Room و DataStore	بله	بله	بله

«بستن توسط کاربر» یعنی Back که Activity را finish می‌کند، کشیدن اپ از Recents، یا force stop.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. کلید را ذخیره کنید، داده را دوباره بسازید

```
class SearchViewModel(
    private val savedStateHandle: SavedStateHandle,
    private val repo: SearchRepository,
) : ViewModel() {

    val query: StateFlow<String> =
        savedStateHandle.getStateFlow(KEY_QUERY, "")

    val results: StateFlow<List<Item>> = query
        .map { q -> repo.search(q) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList(),
        )

    fun onQueryChange(value: String) {
        savedStateHandle[KEY_QUERY] = value
    }

    private companion object { const val KEY_QUERY = "query" }
}
```

```
// Compose: element-level UI state
var expanded by rememberSaveable { mutableStateOf(false) }
```

خود query (چند بایت) ذخیره می‌شود؛ نتیجه‌ها (که ممکن است بزرگ باشند) بعد از process death از repository دوباره ساخته می‌شوند.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۳. دو تله‌ای که همه در آن می‌افتند

تلهٔ مصاحبه

«`onSaveInstanceState` *از process death عبور نمی‌کند.*» — **غلط است.** عبور از process deathی که سیستم انجام می‌دهد دقیقاً کار اصلی آن است. فقط وقتی دور ریخته می‌شود که **کاربر** خودش task را ببندد.

محدودیت حجم

saved state از Binder عبور می‌کند. بافر تراکنش حدود **۱ مگابایت است و بین کل پروسه مشترک است**، پس saved state را کوچک نگه دارید (حداکثر چند ده کیلوبایت). سرریز به `TransactionTooLargeException` ختم می‌شود. id ذخیره کنید، هرگز لیست یا bitmap.

سؤال تکمیلی

«`SavedStateHandle` *بعد از process death داده‌اش را از کجا می‌آورد؟*» — از همان `Bundle` مربوط به saved state که Activity یا back-stack entry ذخیره کرده بود. برای یک Activity، مقادیر پیش‌فرضش شامل extraهای `Intent` هم می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

نشت حافظه، onStop در برابر onDestroy، Navigation و نتیجه‌ها، taskها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. نشت Activity امروز از کجا می‌آید؟

نشت یعنی شیئی با **عمر طولانی‌تر** به شیئی با عمر **کوتاه‌تر** ارجاع نگه دارد: یک Activity، یک View، یا view یک Fragment.

منابع واقعی نشت در کد مدرن:

- یک singleton یا وابستگی `@Singleton` که **context اکتیویتی** یا یک `View` را نگه دارد
- یک `ViewModel` که یک `View`، یک `Activity` یا lambdaیی را که آن‌ها را capture کرده نگه دارد
- Fragmentی که view binding را بعد از `onDestroyView` نگه دارد — Fragment از view خودش بیشتر عمر می‌کند
- coroutineها در `GlobalScope` یا در scope سفارشی‌ای که هیچ‌کس لغوش نمی‌کند
- listenerهایی که روی سرویس‌های سیستمی یا SDKها ثبت شده‌اند و هیچ‌وقت حذف نمی‌شوند
- `Handler.postDelayed` با یک runnable در صف که Activity را capture کرده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. راه‌حل: کار را به یک طول عمر گره بزنید

```
// Activity
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.uiState.collect(::render)
    }
}

// Fragment: the view's lifecycle, not the Fragment's
viewLifecycleOwner.lifecycleScope.launch { /* same as above */ }

// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()
```

- structured concurrency با `viewModelScope` و `lifecycleScope`
- اشیای بلندعمر `applicationContext` می‌گیرند، هرگز Activity
- تشخیص با **LeakCanary** در buildهای debug و heap dump در Memory Profiler
- `WeakReference` آخرین راه است — معمولاً نشانهٔ این است که طراحی مالکیت اشتباه است

تلهٔ مصاحبه

`launchWhenStarted` منسوخ شده: coroutine را **suspend** می‌کند اما flow بالادستی را زنده نگه می‌دارد، پس تولیدکنندهٔ location یا socket در پس‌زمینه به کارش ادامه می‌دهد. `repeatOnLifecycle` آن را **لغو** می‌کند و دوباره شروع می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. `onStop()` در برابر `onDestroy()`

	`onStop()`	`onDestroy()`
معنی	دیگر دیده نمی‌شود	این نمونه دارد از بین می‌رود
بعدش چه می‌آید	`onRestart` ، `onDestroy` **یا kill شدن پروسه بدون هیچ callbackی**	هیچ چیز
قابل اتکا؟	آخرین callbackی که قبل از kill می‌شود رویش حساب کرد	**نه**
اینجا چه کنیم	ذخیرهٔ پیش‌نویس‌ها، توقف کار UI، آزاد کردن دوربین و playerها	آزاد کردن چیزهایی که این نمونه مالکشان است
چک مفید	—	`isFinishing` در برابر `isChangingConfigurations`

تلهٔ مصاحبه

اندروید برای آزاد کردن حافظه Activityها را تک‌تک destroy **نمی‌کند** — **کل پروسه را kill می‌کند**. جملهٔ «سیستم برای آزاد کردن حافظه `onDestroy` را صدا زد» غلط است: بعد از kill هیچ callbackی دریافت نمی‌کنید، فقط وقتی کاربر برمی‌گردد state بازگردانده می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. رفتن بین صفحه‌ها و پاس دادن داده

قاعده: شناسه پاس بدهید، نه شیء. مقصد خودش شیء را از repository می‌خواند — یک منبع حقیقت و saved state کوچک.

```
// Start
startActivity(
    Intent(this, DetailActivity::class.java)
        .putExtra(EXTRA_ITEM_ID, itemId)
)

// Read. Better: in DetailViewModel,
// savedStateHandle[EXTRA_ITEM_ID] already contains the extras
val itemId = intent.getLongExtra(EXTRA_ITEM_ID, -1L)
```

- اگر مجبورید شیء پاس بدهید: `@Parcelize` به‌جای `Serializable` (بدون reflection، سریع‌تر).
- از Android 13 به بعد متد تایپ‌دار `getParcelableExtra(key, Class)` وجود دارد؛ برای پشتیبانی از نسخه‌های قدیمی‌تر از `IntentCompat` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. اپ single-activity و navigation تایپ‌سیف

```
@Serializable data object Home
@Serializable data class Detail(val itemId: Long)

NavHost(navController, startDestination = Home) {
    composable<Home> {
        HomeScreen(
            onOpen = { id -> navController.navigate(Detail(id)) }
        )
    }
    composable<Detail> { DetailScreen() }
}

class DetailViewModel(handle: SavedStateHandle) : ViewModel() {
    private val itemId = handle.toRoute<Detail>().itemId
}
```

- آرگومان‌ها در زمان کامپایل چک می‌شوند و در saved state مربوط به back-stack entry نگه داشته می‌شوند، پس از process death عبور می‌کنند.
- Safe Args همین مشکل را برای گراف‌های XML و Fragmentها حل می‌کرد.
- **Navigation 3** کتابخانهٔ جدیدتر و Compose-first است: back stack یک state ساده است که مالکش خود شما هستید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. برگرداندن نتیجه

`startActivityForResult` و `onActivityResult` منسوخ شده‌اند. از Activity Result API استفاده کنید:

```
class ProfileActivity : ComponentActivity() {
    private val viewModel: ProfileViewModel by viewModels()

    private val pickPhoto = registerForActivityResult(
        ActivityResultContracts.PickVisualMedia()
    ) { uri: Uri? -> uri?.let(viewModel::onPhotoPicked) }

    fun onChangePhotoClick() {
        pickPhoto.launch(
            PickVisualMediaRequest(
                ActivityResultContracts.PickVisualMedia.ImageOnly
            )
        )
    }
}
```

در Compose از `rememberLauncherForActivityResult(...)` استفاده کنید. ابزار photo picker هم به هیچ دسترسی storage نیاز ندارد.

تلهٔ مصاحبه

launcher را بدون شرط و در زمان مقداردهی اولیه register کنید (تعریفش به‌عنوان property کافی است). register کردن بعد از رسیدن lifecycle به STARTED — مثلاً داخل یک click listener — `IllegalStateException` می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. taskها، launch modeها و onNewIntent

حالت یا flag	رفتار
`standard`	هر بار یک نمونهٔ جدید
`singleTop` / `FLAG_ACTIVITY_SINGLE_TOP`	فقط اگر **بالای** stack باشد دوباره استفاده می‌شود ← `onNewIntent`
`singleTask`	یک نمونه در task خودش؛ Activityهای بالای آن پاک می‌شوند ← `onNewIntent`
`singleInstancePerTask` (از API 31)	همیشه ریشهٔ task خودش؛ می‌تواند در چند task باشد
`singleInstance`	تنها Activity در task خودش
`NEW_TASK` و `CLEAR_TASK`	ریست کلاسیک برای «logout و ورود دوباره»

تلهٔ مصاحبه

در `onNewIntent`، متد `getIntent()` همچنان intent **اصلی** را برمی‌گرداند، مگر اینکه `setIntent(intent)` را صدا بزنید. با `ComponentActivity` می‌توانید از `addOnNewIntentListener` هم استفاده کنید.

سؤال تکمیلی

«با Back از Activity ریشه چه اتفاقی می‌افتد؟» — از Android 12، Back روی Activity ریشهٔ launcher به‌جای finish کردن، task را به پس‌زمینه می‌برد. Activity نگه داشته می‌شود (بدون `onDestroy`)، پس اجرای بعدی یک warm start است.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

کامپوننت‌های lifecycle-aware، زمان startup، بازگردانی state در چند صفحه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. یک resource manager آگاه از lifecycle طراحی کنید

هدف: resource (موقعیت مکانی، socket، سنسور، player) **فقط تا وقتی کار کند که یک مصرف‌کنندهٔ قابل‌مشاهده به آن نیاز دارد**، بدون هیچ کد start و stop در صفحه‌ها.

scope را انتخاب کنید:

- **برای هر صفحه** ← یک `Flow` سرد منتشر کنید؛ مصرف‌کننده‌ها تا وقتی حداقل STARTED هستند collect می‌کنند
- **برای کل اپ** ← `ProcessLifecycleOwner`، یعنی foreground و background در سطح همهٔ Activityها

stateها را آگاهانه انتخاب کنید:

- **بین STARTED و STOPPED** برای هر چیزی که کاربر می‌بیند — در multi-window چند Activity هم‌زمان دیده می‌شوند
- **بین RESUMED و PAUSED** فقط برای منابع انحصاری مثل دوربین. از Android 10 چند Activity می‌توانند هم‌زمان RESUMED باشند؛ برای کاری که فقط به Activity بالایی و دارای focus نیاز دارد از `onTopResumedActivityChanged` استفاده کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. برای هر صفحه: callbackFlow

```
class LocationRepository(
    private val client: FusedLocationProviderClient,
) {
    @RequiresPermission(
        anyOf = [ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION]
    )
    fun updates(intervalMs: Long = 5_000) = callbackFlow<Location> {
        val request = LocationRequest.Builder(
            Priority.PRIORITY_BALANCED_POWER_ACCURACY, intervalMs
        ).build()

        val callback = object : LocationCallback() {
            override fun onLocationResult(result: LocationResult) {
                result.lastLocation?.let { trySend(it) }
            }
        }

        client.requestLocationUpdates(
            request, callback, Looper.getMainLooper()
        )
        awaitClose { client.removeLocationUpdates(callback) }
    }
}
```

این flow با collect شدن شروع می‌شود و وقتی collector لغو شود **در** `awaitClose` **پاک‌سازی می‌کند** — و `repeatOnLifecycle(STARTED)` همین لغو را در `onStop` انجام می‌دهد. بدون ارجاع به Activity، بدون start و stop دستی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. برای کل اپ: ProcessLifecycleOwner

```
class AppForegroundObserver(
    private val socket: RealtimeSocket,
) : DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) = socket.connect()
    override fun onStop(owner: LifecycleOwner) = socket.disconnect()
}

// Application.onCreate()
ProcessLifecycleOwner.get().lifecycle
    .addObserver(AppForegroundObserver(socket))
```

- `ProcessLifecycleOwner` رویداد `ON_STOP` را با کمی تأخیر (حدود ۷۰۰ میلی‌ثانیه) بعد از stop شدن آخرین Activity ارسال می‌کند، تا چرخش صفحه شبیه «رفتن اپ به background» به نظر نرسد. هیچ‌وقت `ON_DESTROY` ارسال نمی‌کند.
- برای تست observer از `TestLifecycleOwner` استفاده کنید؛ این کلاس در `lifecycle-runtime-testing` است.

تلهٔ مصاحبه

`@OnLifecycleEvent` منسوخ شده. از `DefaultLifecycleObserver` یا `LifecycleEventObserver` استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. بهینه‌سازی startup — اول اندازه بگیرید

نوع	چه چیزی از قبل در حافظه است	آستانهٔ «زیاد» در Android vitals
Cold	هیچ چیز — پروسه، `Application` و Activity همه ساخته می‌شوند	۵ ثانیه یا بیشتر
Warm	پروسه زنده است، Activity دوباره ساخته می‌شود (یا از saved state بازگردانده می‌شود)	۲ ثانیه یا بیشتر
Hot	پروسه و Activity زنده‌اند و فقط به جلو آورده می‌شوند	۱٫۵ ثانیه یا بیشتر

- **TTID** (زمان تا اولین نمایش): اولین فریم — خط `Displayed` در Logcat
- **TTFD** (زمان تا نمایش کامل): صفحه واقعاً قابل استفاده است — `reportFullyDrawn()`، یا در Compose `ReportDrawnWhen { items.isNotEmpty() }`
- **ابزارها:** Macrobenchmark (در آزمایشگاه و CI)، traceهای Perfetto (دیگر Systrace قدیمی است)، Android vitals (کاربران واقعی)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. چیزهایی که واقعاً اثر دارند

۱. **Baseline Profiles** (به‌همراه Startup Profiles): مسیرهای حیاتی کد از همان اولین اجرا، از قبل کامپایل می‌شوند. گوگل حدود ۳۰ درصد اجرای سریع‌تر کد در اولین اجرا را گزارش می‌کند.

۲. **فریم اول را بلاک نکنید:** شبکه یا دیسک روی ترد اصلی ممنوع؛ دادهٔ cacheشده یا skeleton نشان دهید.

۳. **مقداردهی تنبل:** `Lazy<T>` یا `Provider<T>` در Hilt، و `by inject()` در Koin. به‌جای ContentProviderهایی که SDKها برای init خودکار دارند از App Startup استفاده کنید، یا init را به زمان نیاز موکول کنید.

۴. **R8** (که حالت full mode آن از AGP 8 پیش‌فرض است) و حذف وابستگی‌های بی‌استفاده.

۵. **SplashScreen API** با `core-splashscreen`: از `setKeepOnScreenCondition` فقط برای یک چک کوتاه و محلی استفاده کنید.

۶. **StrictMode** در buildهای debug، برای گرفتن دسترسی به دیسک و شبکه روی ترد اصلی.

۷. **جلوی regression را بگیرید:** Macrobenchmark را روی دستگاه واقعی در CI اجرا کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. benchmark و profile در کد

```
@RunWith(AndroidJUnit4::class)
class StartupBenchmark {
    @get:Rule val rule = MacrobenchmarkRule()

    @Test fun coldStartup() = rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(StartupTimingMetric()),
        iterations = 10,
        startupMode = StartupMode.COLD,
    ) {
        pressHome()
        startActivityAndWait()
    }
}

@RunWith(AndroidJUnit4::class)
class BaselineProfileGenerator {
    @get:Rule val rule = BaselineProfileRule()

    @Test fun generate() = rule.collect("com.example.app") {
        pressHome()
        startActivityAndWait()
        // + the critical user journey: open list, scroll, open detail
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. بازگردانی state در چندین صفحه

- **یک منبع حقیقت** (Room یا cache). صفحه‌ها state خود را از آن دوباره می‌سازند؛ کپی خصوصی نگه نمی‌دارند.
- **هر مقصد ViewModel خودش را دارد** و فقط *کلیدها* را در `SavedStateHandle` ذخیره می‌کند.
- **بگذارید navigation خودش back stack را ذخیره کند.** `NavController` و `FragmentManager` خود back stack و saved state هر entry را پایدار نگه می‌دارند.
- bottom navigation با **چند back stack**:

```
navController.navigate(tab.route) {
    popUpTo(navController.graph.findStartDestination().id) {
        saveState = true
    }
    launchSingleTop = true
    restoreState = true
}
```

- Fragmentها: با `viewLifecycleOwner` observe کنید. `replace` همراه با back stack، state ویوی قبلی را ذخیره و viewهایش را آزاد می‌کند؛ `show` و `hide` همهٔ viewها را در حافظه نگه می‌دارد — فقط برای تب‌های سبکی که زیاد بین‌شان جابه‌جا می‌شوید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. واقعاً تستش کنید

روش	چه چیزی را شبیه‌سازی می‌کند
چرخاندن صفحه یا `ActivityScenario.recreate()`	configuration change
گزینهٔ «Don't keep activities» در Developer options	از بین رفتن Activity — **پروسه زنده می‌ماند**
اپ را به background ببرید، `adb shell am kill <package>` بزنید، بعد از Recents بازش کنید	**process death واقعی**
`StateRestorationTester` در Compose	`rememberSaveable` در تست

تلهٔ مصاحبه

گزینهٔ «Don't keep activities» شبیه‌ساز process death **نیست**: singletonها و cacheهای داخل حافظه زنده می‌مانند، پس بدترین باگ‌ها پنهان می‌مانند. روی اپی که در background است از `am kill` استفاده کنید.

سؤال تکمیلی

«*جلوی regression را چطور می‌گیرید؟*» — برای هر صفحهٔ حیاتی یک instrumented test که آن را از saved state دوباره می‌سازد، به‌علاوهٔ یک دور `am kill` دستی در چک‌لیست release.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

مانیتورینگ در کل اپ، streamهای real-time، بقای پروسه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. مانیتورینگ کارایی lifecycle در کل اپ

۱. instrumentation داخل اپ — سربار کم، به‌صورت نمونه‌برداری

- `ActivityLifecycleCallbacks` و `FragmentLifecycleCallbacks` ← زمان‌بندی هر صفحه
- بخش‌های `androidx.tracing` ← در Perfetto و Macrobenchmark دیده می‌شوند
- **JankStats** ← فریم‌های janky با برچسب صفحهٔ فعلی
- **`ApplicationExitInfo`** (از API 30) ← *چرا* پروسهٔ قبلی مرد

۲. انتقال داده — batch کردن محلی، آپلود با WorkManager، نمونه‌برداری در هر release، بدون PII

۳. محیط واقعی و آزمایشگاه

- در محیط واقعی: Android vitals، Firebase Performance یا Crashlytics، یا backend خودتان
- در آزمایشگاه: Macrobenchmark روی دستگاه‌های فیزیکی در CI، که با regression در P50 یا P90 شکست می‌خورد

۴. داشبورد و هشدار — تفکیک بر اساس نسخهٔ اپ، نسخهٔ OS و ردهٔ دستگاه؛ هشدار روی regressionها در طول staged rollout



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. نقطه‌های جمع‌آوری داده

```
class ScreenTimingCallbacks(
    private val sink: MetricsSink,
) : Application.ActivityLifecycleCallbacks {

    private val started = HashMap<Int, Long>()
    private val Activity.key get() = System.identityHashCode(this)
    private fun now() = SystemClock.elapsedRealtime()

    override fun onActivityPreCreated(a: Activity, state: Bundle?) {
        started[a.key] = now()
    }

    override fun onActivityResumed(a: Activity) {
        val start = started.remove(a.key) ?: return
        sink.record(
            name = "create_to_resume",
            screen = a.javaClass.simpleName,
            millis = now() - start,
        )
    }
    // remaining callbacks: empty overrides
}
```

`create_to_resume` یک شاخص تقریبی و ارزان است. برای اولین فریم واقعی، به TTID و TTFD از Macrobenchmark و Android vitals اعتماد کنید.



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۱. بفهمید پروسه چرا مرد

```
fun reportPreviousExits(context: Context, store: ExitStore) {
    val am = context.getSystemService(ActivityManager::class.java)
    am.getHistoricalProcessExitReasons(context.packageName, 0, 10)
        .filter { it.timestamp > store.lastReportedTimestamp }
        .forEach { exit ->
            store.report(
                reason = exit.reason, // LOW_MEMORY, ANR, CRASH…
                importance = exit.importance, // fg or cached?
                pssKb = exit.pss,
                description = exit.description,
            )
        }
}
```

اینکه پروسه‌های **cached** به‌خاطر کمبود حافظه kill شوند عادی است. یک جهش بعد از یک release، یا kill شدن در حالی که اپ در **foreground** است، سیگنال واقعی است.

از ANDROID 15

`ProfilingManager` به اپ اجازه می‌دهد heap dump، stack sampling یا trace سیستمی را از دستگاه کاربران واقعی درخواست کند، با محدودیت نرخی که سیستم تعیین می‌کند — trace واقعی از میدان به‌جای حدس و گمان.



اسماعیل مرادی
@esmaeeil-moradi700



سؤال ۱۲. دادهٔ real-time آگاه از lifecycle

یک صفحهٔ قیمت فقط تا وقتی دیده می‌شود به فید زندهٔ WebSocket نیاز دارد — بدون اینکه با هر چرخش صفحه دوباره وصل شود.

```
// Money is never a Double
data class Price(val symbol: String, val value: BigDecimal)

class PriceViewModel(
    repo: PriceRepository,
    savedStateHandle: SavedStateHandle,
) : ViewModel() {
    private val symbol: String =
        checkNotNull(savedStateHandle["symbol"])

    val state: StateFlow<PriceUiState> = repo.prices(symbol)
        .map<Price, PriceUiState> { PriceUiState.Live(it) }
        .retryWhen { cause, attempt ->
            val retry = cause is IOException
            if (retry) delay(backoffMs(attempt))
            retry
        }
        .catch { emit(PriceUiState.Error) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = PriceUiState.Loading,
        )
}

// UI
val state by viewModel.state.collectAsStateWithLifecycle()
```



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱۲. چرا این طراحی برنده است

- **چرخش صفحه:** UI جدید در کمتر از ۵ ثانیه subscribe می‌کند، پس `WhileSubscribed(5_000)` سوکت را باز نگه می‌دارد — بدون اتصال دوباره.
- **پس‌زمینه:** collect در `onStop` متوقف می‌شود؛ ۵ ثانیه بعد جریان بالادستی لغو می‌شود و `awaitClose` سوکت را می‌بندد.
- **تاب‌آوری:** exponential backoff روی خطاهای شبکه؛ خطاهای دیگر به‌جای کرش، به UI state تبدیل می‌شوند.
- **تست‌پذیری:** repository یک `Flow` ساده است — با یک سوکت fake و Turbine تستش کنید، بدون درگیر شدن با lifecycle.
- **نوع درست برای پول:** `BigDecimal` (یا واحدهای خرد به شکل `Long`)، هرگز `Double`.

پس کِی هنوز LIFECYCLEOBSERVER می‌نویسیم؟

برای APIهایی که stream نیستند: دوربین و media playerها، SDKهای third-party با `start()` و `stop()`، ردیابی صفحه در analytics — و دغدغه‌های سطح اپ از طریق `ProcessLifecycleOwner`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. مدیریت Activity در برابر بقای پروسه

سیستم چطور تصمیم می‌گیرد:

- اهمیت پروسه، از زیاد به کم: **foreground**، **visible**، **perceptible** یا **service**، **cached**
- `lmkd` (در userspace، و از Android 10 مبتنی بر فشار حافظه از طریق PSI) بر اساس `oom_score_adj` پروسه‌ها را kill می‌کند — **اول پروسه‌های cached**
- اندروید مدرن پروسه‌های cached را **freeze** هم می‌کند تا هیچ CPUیی مصرف نکنند
- سیستم **پروسه‌ها** را kill می‌کند، نه Activityها را تک‌تک

چه چیزی شانس شما را بیشتر می‌کند:

- در background سبک باشید: cacheها و bitmapها را در `onTrimMemory(TRIM_MEMORY_UI_HIDDEN)` رها کنید
- بدون نشت — Activityهای نشت‌کرده یعنی heap بزرگ‌تر و kill زودتر
- اما **طوری طراحی کنید که انگار kill خواهید شد** — چون خواهید شد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. باگ شمارهٔ ۱ در process death

کاربر در صفحهٔ Checkout است ← اپ به background می‌رود ← پروسه kill می‌شود ← کاربر برمی‌گردد.

اندروید **فقط Activity مربوط به Checkout** را با saved state خودش بازمی‌گرداند. `Application.onCreate` دوباره اجرا می‌شود، اما صفحه‌های **قبل از** Checkout هیچ‌وقت در این پروسه اجرا نشده‌اند.

```
// BAD: SessionHolder was filled by LoginActivity.
// After process death it's empty -> crash.
val user = SessionHolder.user!!

// GOOD: re-derive from the source of truth (DataStore / Room)
val user: StateFlow<User?> = userRepository.currentUser
    .stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = null,
    )
```

قاعده: هر صفحه باید بتواند فقط با کلیدهای ذخیره‌شده‌اش و repository از صفر شروع شود. هیچ صفحه‌ای نباید به state داخل حافظه‌ای تکیه کند که صفحهٔ قبلی تنظیمش کرده.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. کاری که باید بیشتر از UI عمر کند

نیاز	ابزار
قابل تعویق اما تضمین‌شده (همگام‌سازی، آپلود)	**WorkManager** — پایدار، با retry، از process death و reboot عبور می‌کند
انتقال حجیمی که کاربر شروعش کرده (Android 14 به بعد)	**User-initiated data transfer job**
مداوم و قابل‌مشاهده برای کاربر (پخش، مسیریابی، تماس)	**Foreground service** با type اعلام‌شده

محدودیت‌هایی که ارزش دارد در مصاحبه نام ببرید:

- از Android 12 به بعد: شروع foreground service **از background** در بیشتر موارد مسدود است
- از Android 14 به بعد: هر foreground service به یک `foregroundServiceType` و permission متناظر نیاز دارد
- از Android 15 به بعد: سرویس‌های `dataSync` و `mediaProcessing` **محدودیت زمانی** دارند (۶ ساعت در هر ۲۴ ساعت)



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱ — خلاصهٔ یک‌صفحه‌ای

- به **state** فکر کنید: کار UI را تا وقتی حداقل STARTED هستید انجام دهید.
- `onPause` باید سریع باشد؛ `onStop` آخرین callback قابل اتکاست؛ `onDestroy` ممکن است هیچ‌وقت اجرا نشود.
- دیالوگ‌ها Activity را pause نمی‌کنند — Activityهای دیگر می‌کنند.
- ViewModel از چرخش صفحه عبور می‌کند، **نه** از process death.
- saved state از process death عبور می‌کند، **نه** از بستن task توسط کاربر. کوچک نگهش دارید.
- id پاس بدهید، نه شیء. داده را از repository دوباره بسازید.
- `repeatOnLifecycle` یا `collectAsStateWithLifecycle` — هرگز `launchWhenX`.
- گزینهٔ «Don't keep activities» با process death یکی نیست. با `am kill` تست کنید.
- زمان راه‌اندازی: TTID و TTFD، Baseline Profiles، و Macrobenchmark در CI.
- هر صفحه باید بتواند فقط با کلیدهای ذخیره‌شده و repository از صفر شروع شود.

روز ۲ در راه است ← این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700